Goal: Get comfortable with PyTorch tensors

In [1]:
```
#version 2024 with cuda / GPU

import numpy as np
import torch #no "as t" for pedagogic reasons to be aware for torch
import time

print(torch.__version__)
print(torch.cuda.device_count())
if(torch.cuda.device_count()):
    print("GPU available")
```

```
1.12.1
0
```

In [2]:
```
#fill your first tensor, take numpy array as starting point
#use l_vars as naming convention to signal data type
#note that arrays and tensors are C++ objects
#Python and PyTorch are written in C++
#tensors try to mirror arrays as far as possible

a_c = np.array([0.5,  14.0, 15.0], dtype='float32')#use float cause GPU slow down with 64bit
print(a_c[2])#usual indexing
a_u = np.array([48.4, 60.4, 68.4], dtype='float32')
a_z = np.zeros((1,3), dtype='float32')#such constructors re-appear later

#we want to build data matrix and the arrays shall become columns

#we transpose and create tensor from array, could also use torch.from_numpy(a_c)

t_c = torch.t(torch.tensor(a_c))
print("\n Welcome to PyTorch")
print(t_c[2])#note the diff! hence to zoom in need
print(t_c[2].numpy())#numpy gives back a numpy array

t_u = torch.t(torch.tensor(a_u))
t_z = torch.t(torch.tensor(a_z))
#check shape
print(t_c.shape, t_z.shape, t_u.shape)
t_u.dim()
```

```
15.0

 Welcome to PyTorch
tensor(15.)
15.0
torch.Size([3]) torch.Size([3, 1]) torch.Size([3])
1
```

Out[2]:

In [3]:
```
#CUDA (Compute Unified Device Architecture) is a programming model
#and parallel computing platform developed by Nvidia. Using CUDA,
#one can maximize the utilization of Nvidia-provided GPUs

if(torch.cuda.device_count()):
    #create tensor on GPU
    t_gpu_c = torch.tensor(a_c, device = "cuda")
    print(t_c)
    print(t_gpu_c)
    #cuda:0 counts the GPUs, if only one c=0, else 0,1,2
    print("\n")
    #alternative route copy tensor to tensor_gpu
    t_gpu_u = t_u.to(device = "cuda")
    print(t_u)
    print(t_gpu_u)
    #and the return ticket
    t_uu = t_gpu_u.to(device = "cpu")
    print(t_uu)
    print("done with cuda")
```

In [4]:
```
#length or random tensor
torch.manual_seed(2023)

t_test = torch.randint(0, 5, (3,))
print(t_test)
print(t_test**2)

if(torch.cuda.device_count()):
    for n in range(1,10):

        t_r = torch.randint(0, 5, (n*1000000,))
        t_gpu_r = t_r.to(device = "cuda")

        start_time = time.time()
        t_r ** 2
```

```
        print("Dimension des quad. Vektors", n*1000000)
        print('---Rechenzeit CPU: %s seconds---' % (time.time() - start_time))

        start_time = time.time()
        t_gpu_r ** 2
        print('---Rechenzeit GPU: %s seconds---' % (time.time() - start_time))
        print("\n\n")
```

```
tensor([4, 4, 2])
tensor([16, 16,  4])
```

In [5]:
```
# Starte Zeitmessung für die Laufzeit
```

In [6]:
```
#align shapes, perform this only once!!!
t_cc = t_c.clone().detach().unsqueeze(1)#clone creates copy in different memory location
t_uc = t_u.clone().detach().unsqueeze(1)
#check shape
print(t_cc.shape, t_z.shape, t_uc.shape)

t_m = torch.cat((t_cc,t_uc,t_z),1)

print(t_m)
```

```
torch.Size([3, 1]) torch.Size([3, 1]) torch.Size([3, 1])
tensor([[ 0.5000, 48.4000,  0.0000],
        [14.0000, 60.4000,  0.0000],
        [15.0000, 68.4000,  0.0000]])
```

In [7]:
```
#play with index

print("Eine Zelle", t_m[1,1],"\n Erste Spalte",t_m[:,1],"\n Ausschnitt",t_m[:,::2])
print("\n Abfragen", t_m[t_m>50])

#how is it stored in RAM?

print("all elements are stored in a sequence starting at", t_m.data_ptr())
print("if u want to go to beginning of next line move right in RAM x times. x =", t_m.stride()[0])

#enforce that RAM storage contains no gaps
#t_m = t_m.contiguous
```

```
#print(t_m)#gives back adress in hexadecimal eg 0x000001F2EBE0DFD0, in decimal 127723
#use https://bin-dez-hex-umrechner.de/
```

```
Eine Zelle tensor(60.4000)
 Erste Spalte tensor([48.4000, 60.4000, 68.4000])
 Ausschnitt tensor([[ 0.5000,  0.0000],
        [14.0000,  0.0000],
        [15.0000,  0.0000]])

 Abfragen tensor([60.4000, 68.4000])
all elements are stored in a sequence starting at 3056208838208
if u want to go to beginning of next line move right in RAM x times. x = 3
```

In [8]:
```
#saving in default working dir of Jupyter, serialisation
torch.save(t_m, "save_t_m.pt")#use pt and not pth! Latter one collides with python path
t_ml = torch.load("save_t_m.pt", map_location = 'cpu')#avoid default access to GPU
print(t_ml)
```

```
tensor([[ 0.5000, 48.4000,  0.0000],
        [14.0000, 60.4000,  0.0000],
        [15.0000, 68.4000,  0.0000]])
```

In [9]:
```
#compute, apply methods of tensor to it, names like in Numpy
torch.tanh(t_m)
```

Out[9]:
```
tensor([[0.4621, 1.0000, 0.0000],
        [1.0000, 1.0000, 0.0000],
        [1.0000, 1.0000, 0.0000]])
```

In [10]:
```
#show grad feature wrt last operation
t_x = torch.tensor([10.0], requires_grad = True)
t_x = t_x.pow(2)
print(t_x)
t_x = t_x.sqrt()
print(t_x)
print(t_x.detach().numpy())
print("\nXXX DONE XXX")
```

```
tensor([100.], grad_fn=<PowBackward0>)
tensor([10.], grad_fn=<SqrtBackward0>)
[10.]

XXX DONE XXX
```

In [ ]: